# Soundscape

Music Room Application

DA219B Fullstack Lab

Lawrence Arciniegas

April 23, 2026

## Contents

# 1. System Overview

Soundscape is a web application that aims to solve one problem: letting people listen to the same music at the same time and stay sync. It is designed so that you join a room and just jam to your favorite songs with a cool interphase that foucus on making it a visual experience. Although other apps like Spotify offers similar options, Soundscape is focused on the social aspect of music listening and provides an inmersive experience.

The app has two sides: the front end (React) is the visual interface where users can create rooms, add songs to the queue, and see what others are listening to. The back end (Express.js) handles all the logic for managing rooms, songs, and queues, as well as real-time synchronization using Socket.IO. MongoDB is used to store all the data related to songs, rooms, and queues.

## 1.1. Technology Stack

- **Frontend:** React 18 with Vite, Socket.IO client for real-time communication
- **Backend:** Express.js with Node.js, MongoDB via Mongoose ODM
- **Database:** MongoDB Atlas (cloud-hosted)
- **Real-time Communication:** Socket.IO for WebSocket support
- **Authentication:** Environment variables for secrets (.env)
- **Process Management:** concurrently for unified startup

# 2. Database Design

The database consists of three primary collections with relationships managed through MongoDB ObjectId references. This design allows correct functionality of room management, song queuing, and real-time synchronization.

## 2.1. Entity Relationship Diagram

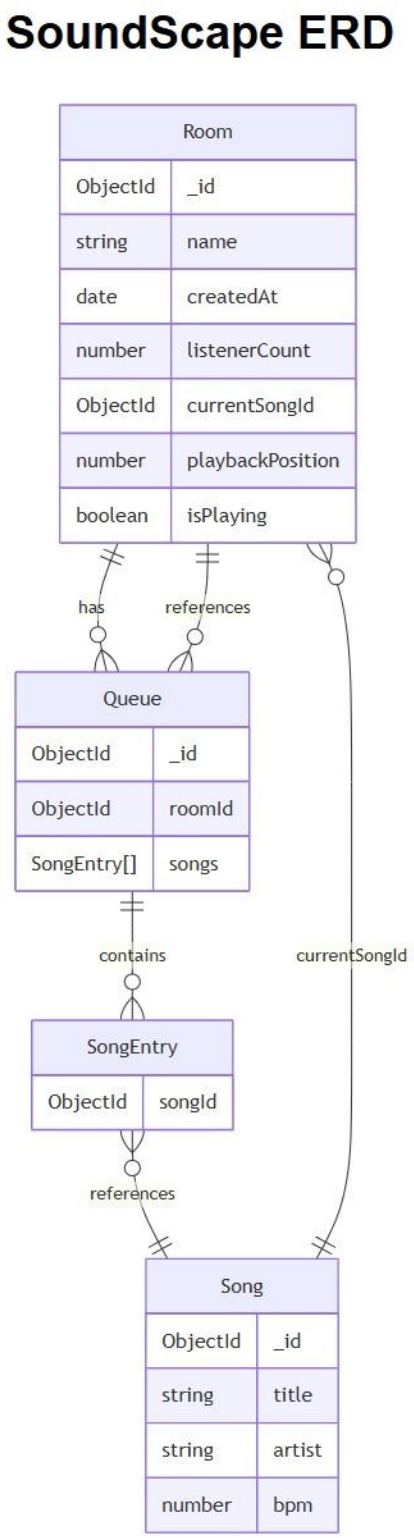


Figure 1: Entity Relationship Diagram for Soundscape

## 2.2. Collection: Songs

```
{
  "_id": ObjectId("..."),
  "title": "Blinding Lights",
  "artist": "The Weeknd",
  "bpm": 171,
  "audioUrl": "/audio/blinding-lights.mp3",
  "duration": 200,
  "genre": "Synthpop",
  "coverColor": "#E83A5C"
}
```

Listing 1: Song Document Structure

**Purpose:** Stores individual songs with metadata.
**Custom Fields:**

- `bpm` (Beats Per Minute): Domain-specific metric for music characterization
- `genre`: Categorizes songs by musical style
- `coverColor`: Dynamic visual representation tied to song characteristics

**Validation:** Title and artist are required. BPM constrained to 40–300 range (realistic music range).
**Sample Data:** 7 realistic songs including songs like: "Blinding Lights" (The Weeknd), "Heat Waves" (Glass Animals), "Shivers" (Ed Sheeran).

### 2.3. Collection: Rooms

```
{
  "_id": ObjectId("..."),
  "name": "Late Night Vibes",
  "createdAt": ISODate("2024-04-20T10:30:00Z"),
  "listenerCount": 5,
  "currentSongId": ObjectId("..."),
  "playbackPosition": 45,
  "isPlaying": true
}
```

Listing 2: Room Document Structure

**Purpose:** Represents collaborative music rooms where users gather.
**Relationships:**

- `currentSongId` references the Song currently playing (nullable)
- Linked to Queue collection for song ordering

**Custom Fields:**

- `playbackPosition`: Tracks synchronization point across clients
- `isPlaying`: Boolean state for pause/play control

**Validation:** Room name 2–50 characters, must be unique.
**Sample Data:** "Late Night Vibes", "Workout Mode", "Chill Lounge".

### 2.4. Collection: Queues

```
{
  "_id": ObjectId("..."),
  "roomId": ObjectId("..."),
  "songs": [
    {
      "songId": ObjectId("..."),
      "addedAt": ISODate("2024-04-20T10:35:00Z"),
      "votes": 2
    },
    {
      "songId": ObjectId("..."),
      "addedAt": ISODate("2024-04-20T10:37:00Z"),
      "votes": 0
    }
  ]
}
```

Listing 3: Queue Document Structure

**Purpose:** Maintains ordered song queues per room with voting metadata.
**Relationships:**

- `roomId`: References the parent Room (required)
- `songs[].songId`: References each Song in queue

**Custom Fields:**

- `votes`: Allows democratic song promotion (future feature)

**Validation:** roomId is required and must be a valid ObjectId.
**Sample Data:** 3 rooms with 3 realistic songs queued per room.

### 2.5. Database Relationships

| Relationship | From | To | Type |
|---|---|---|---|
| Current Track | Room.currentSongId | Song._id | One-to-One |
| Room Queue | Queue.roomId | Room._id | Many-to-One |
| Queue Songs | Queue.songs[].songId | Song._id | Many-to-Many |

Table 1: Database Relationships

## 3. API Endpoints

The API follows RESTful conventions with proper HTTP status codes, error handling, and populated references for efficient data retrieval.

### 3.1. Endpoint 1: GET /api/rooms/:id

**Description:** Retrieve a single room with its current song populated.

### 3.1.1 Request

```
GET /api/rooms/507f1f77bcf86cd799439011 HTTP/1.1
Host: localhost:5000
Accept: application/json
```

### 3.1.2 Response (200 OK)

```
{
  "_id": "507f1f77bcf86cd799439011",
  "name": "Late Night Vibes",
  "createdAt": "2024-04-20T10:30:00.000Z",
  "listenerCount": 5,
  "currentSongId": {
    "_id": "507f1f77bcf86cd799439012",
    "title": "Blinding Lights",
    "artist": "The Weeknd",
    "bpm": 171,
    "audioUrl": "/audio/blinding-lights.mp3",
    "duration": 200,
    "genre": "Synthpop",
    "coverColor": "#E83A5C"
  },
  "playbackPosition": 125,
  "isPlaying": true
}
```

### 3.1.3 Response (404 Not Found)

```
{
  "error": "Room not found"
}
```

### 3.1.4 Implementation Flow

1. **Route Handler:** `router.get('/:id')` receives request
2. **Controller Logic:** `getRoom` extracts ID from URL params
3. **Database Query:** `Room.findById(id).populate('currentSongId')`
4. **Error Handling:** If room not found, returns 404; catch block returns 500
5. **Response:** Populated room document sent as JSON

### 3.1.5 Code Reference

From `controllers/roomController.js`:

```
exports.getRoom = async (req, res) => {
  try {
    const room = await Room.findById(req.params.id)
      .populate('currentSongId');
    if (!room)
      return res.status(404).json({ error: 'Room not found' });
```

```
    res.json(room);
  } catch (err) {
    res.status(500).json({ error: 'Failed to fetch room' });
  }
};
```

## 3.2. Endpoint 2: POST /api/rooms/:roomId/queue

**Description:** Add a song to a room's queue with validation.

### 3.2.1 Request

```
POST /api/rooms/507f1f77bcf86cd799439011/queue HTTP/1.1
Host: localhost:5000
Content-Type: application/json

{
  "songId": "507f1f77bcf86cd799439012"
}
```

### 3.2.2 Response (200 OK)

```
{
  "_id": "507f1f77bcf86cd799439020",
  "roomId": "507f1f77bcf86cd799439011",
  "songs": [
    {
      "_id": "63a7f8e1d5f4e2a1b8c2d3e4",
      "songId": {
        "_id": "507f1f77bcf86cd799439015",
        "title": "Stay",
        "artist": "Justin Bieber",
        "bpm": 105,
        "duration": 138,
        "genre": "Pop"
      },
      "addedAt": "2024-04-20T10:35:00.000Z",
      "votes": 0
    },
    {
      "_id": "63a7f8e1d5f4e2a1b8c2d3e5",
      "songId": {
        "_id": "507f1f77bcf86cd799439012",
        "title": "Blinding Lights",
        "artist": "The Weeknd",
        "bpm": 171,
        "duration": 200,
        "genre": "Synthpop"
      },
      "addedAt": "2024-04-20T10:38:30.000Z",
      "votes": 0
    }
  ]
}
```

### 3.2.3 Error Responses

```
{
  "error": "songId is required"
}
```

Listing 4: 400 Bad Request - Missing songId

```
{
  "error": "Invalid songId format"
}
```

Listing 5: 400 Bad Request - Invalid ObjectId

```
{
  "error": "Song not found"
}
```

Listing 6: 404 Not Found - Song doesn't exist

### 3.2.4 Implementation Flow

1. **Validation:** Check that `songId` is provided
2. **Format Check:** Validate `songId` is valid MongoDB ObjectId
3. **Database Lookup:** Verify song exists
4. **Queue Update:** Push song entry to queue.songs array
5. **Populate References:** Return populated queue with full song data
6. **Error Handling:** Return appropriate HTTP codes (400, 404, 500)

### 3.2.5 Code Reference

From `controllers/queueController.js`:

```
exports.addToQueue = async (req, res) => {
  try {
    const { songId } = req.body;
    if (!songId)
      return res.status(400).json({ error: 'songId is required' });
    if (!mongoose.Types.ObjectId.isValid(songId)) {
      return res.status(400).json({ error: 'Invalid songId format' });
    }
    const song = await Song.findById(songId);
    if (!song)
      return res.status(404).json({ error: 'Song not found' });
    const queue = await Queue.findOneAndUpdate(
      { roomId: req.params.roomId },
      { $push: { songs: { songId } } },
      { new: true }
    ).populate('songs.songId');
    res.json(queue);
  } catch (err) {
    res.status(500).json({ error: 'Failed to add to queue' });
  }
};
```

## 3.3. Endpoint 3: PUT /api/rooms/:id

**Description:** Update room properties including name with duplicate checking and inline edit UI support.

### 3.3.1 Request

```
PUT /api/rooms/507f1f77bcf86cd799439011 HTTP/1.1
Host: localhost:5000
Content-Type: application/json

{
  "name": "Epic Movie Night"
}
```

### 3.3.2 Response (200 OK)

```
{
  "_id": "507f1f77bcf86cd799439011",
  "name": "Epic Movie Night",
  "createdAt": "2024-04-20T10:30:00.000Z",
  "listenerCount": 5,
  "currentSongId": "507f1f77bcf86cd799439012",
  "playbackPosition": 125,
  "isPlaying": true
}
```

### 3.3.3 Error Responses

```
{
  "error": "Room name is required"
}
```

Listing 7: 400 Bad Request - Missing name

```
{
  "error": "Room name already taken"
}
```

Listing 8: 409 Conflict - Duplicate name

```
{
  "error": "Room not found"
}
```

Listing 9: 404 Not Found

### 3.3.4 Implementation Flow

1. **Validation:** Check name field is provided and non-empty
2. **Room Lookup:** Find room by ID from URL parameters
3. **Duplicate Check:** Query database for existing rooms with same name

4. **Conflict Detection:** Only allow duplicate if it's the same room being updated
5. **Update Operation:** Modify room.name and save to database
6. **Response:** Return updated room document

#### 3.3.5 Code Reference

From `controllers/roomController.js`:

```
exports.updateRoom = async (req, res) => {
  try {
    const { name } = req.body;
    if (!name)
      return res.status(400).json({ error: 'Room name is required' });
    const room = await Room.findById(req.params.id);
    if (!room)
      return res.status(404).json({ error: 'Room not found' });
    // Check for duplicate name
    const existing = await Room.findOne({ name });
    if (existing && existing._id.toString() !== req.params.id) {
      return res.status(409).json({ error: 'Room name already taken' })
    ;
    }
    room.name = name;
    await room.save();
    res.json(room);
  } catch (err) {
    res.status(500).json({ error: 'Failed to update room' });
  }
};
```

**Frontend Integration:** The UI provides inline editing via controlled input in the room list, allowing users to rename rooms without page navigation. Changes are sent via PUT request and immediately reflected in the UI upon success.

## 4. Reflection: Development Challenges and Solutions

### 4.1. Challenge: MongoDB Atlas Connection via DNS SRV Records

**Problem:** The application initially used the 'mongodb+srv://' URI scheme for MongoDB Atlas connectivity. During deployment testing, connections failed with 'ECONNREFUSED' errors despite correct credentials.

**Root Cause:** DNS SRV record resolution failed on the target network. The 'mongodb+srv://' URI relies on DNS SRV records to discover and connect to MongoDB Atlas cluster nodes. When the DNS server did not return SRV records, the driver could not resolve cluster node addresses. This issue was discovered during initial MongoDB setup (Commit 156b5d82, April 8).

**Solution:** Decided to switch from SRV-based URI to explicit connection string with all node addresses:

```
mongodb+srv://mongo:1404@soundscape.icmlyru.mongodb.net/?appName=
    soundscape
```

Listing 10: Connection String Before (SRV)

```
mongodb://mongo:1404@ac-zgqzfqs-shard-00-00.icmlyru.mongodb.net:27017,\
ac-zgqzfqs-shard-00-01.icmlyru.mongodb.net:27017,\
```

```
ac-zgqzfqs-shard-00-02.icmlyru.mongodb.net:27017/?ssl=true&\
replicaSet=atlas-61fuhw-shard-0&authSource=admin&appName=soundscape
```

Listing 11: Connection String After (Explicit Nodes)

**Implementation:** Updated '.env' file with the new connection string. Connection string documented in Commit 156b5d82 with notation: "connected to mongo (using standard connection string as SRV did not work, took some hours of troubleshoot)".

**Lessons Learned:** On MongoDB Atlas, the 'mongodb+srv://' URI is convenient but can lead to connectivity issues if DNS resolution fails. Using explicit node addresses in the connection string provides a solution to this problem. Always test database connectivity in the target deployment environment early to catch such issues.

# 5. Feature Iteration and Improvements

## 5.1. Initial Implementation

The queue system initially stored all songs as flat references without metadata. Additions to the queue were simple array pushes with no tracking of when songs were added or user voting.

```
const queueSchema = new mongoose.Schema({
  roomId: mongoose.Schema.Types.ObjectId,
  songs: [mongoose.Schema.Types.ObjectId]
});
```

Listing 12: Early Queue Model (April 8 Initial Backend Setup)

**Issues:**

- No way to track insertion order with meaningful metadata
- Cannot implement voting mechanisms for songs
- Lost information about who added what and when
- Difficult to implement removal by queue position

## 5.2. Improved Implementation

The queue schema was refactored to include an embedded document for each queue entry, capturing metadata crucial for domain-specific features.

```
const queueSchema = new mongoose.Schema({
  roomId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Room',
    required: true
  },
  songs: [{
    songId: { type: mongoose.Schema.Types.ObjectId, ref: 'Song' },
    addedAt: { type: Date, default: Date.now },
    votes: { type: Number, default: 0 }
  }]
});
```

Listing 13: Enhanced Queue Model (April 20 API Improvements - Commit ee3146c6)

**Improvements:**

- `addedAt` timestamp enables chronological ordering and UI displays
- `votes` field supports democratic song promotion
- Structured as embedded documents for atomic operations
- Enables sorting and filtering by addition time or vote count

### 5.3. Git History Reference

- **Commit e6af04ce** (Apr 8): "front end views created and socket, also package.json change in the backend"
- **Commit ee3146c6** (Apr 20): "feat: PUT /api/rooms/:id to update the room name, ObjectId validation, edit UI"

**Visible Iteration:** The 12-day development period between initial backend setup and API improvements demonstrates iterative refinement. Early commits did not follow the correct fix, feat format butshow how the process evolved. This progression shows the evolution of the projectrather than a single dump of finished code.

### 5.4. Improvement: ObjectId Validation on Queue Additions

**Initial Issue:** When users submitted malformed MongoDB ObjectIds to the 'POST /api/-rooms/:roomId/queue' endpoint, Mongoose threw a 'CastError' during database operations. The generic catch block then returned HTTP 500 (Internal Server Error) instead of the semantically correct HTTP 400 (Bad Request).

**Before:**

```
const { songId } = req.body;
if (!songId)
  return res.status(400).json({ error: 'songId is required' });
// Missing: ObjectId format validation
const song = await Song.findById(songId);  // Throws CastError if
    invalid
```

**After:**

```
const { songId } = req.body;
if (!songId)
  return res.status(400).json({ error: 'songId is required' });
if (!mongoose.Types.ObjectId.isValid(songId)) {
  return res.status(400).json({ error: 'Invalid songId format' });
}
const song = await Song.findById(songId);
```

**Impact:**

- Prevents unhandled exceptions from propagating
- Returns correct HTTP 400 status for client errors
- Provides clear error message for debugging
- Improves API reliability and client experience

**Pattern Applied:** This validation pattern was added to all routes accepting ObjectId input, ensuring consistent error handling across the API. Implemented in Commit ee3146c6 along with PUT endpoint for room updates and edit UI integration.

# A. Additional Resources

## A.1. Setup Commands

**One-command startup:**

```
npm run dev  # Runs backend and frontend concurrently
```

**Manual setup:**

```
# Terminal 1: Backend
cd backend
npm install
npm run dev

# Terminal 2: Frontend
cd frontend
npm install
npm run dev

# Terminal 3: Seed database (optional)
cd backend
npm run seed
```

## A.2. Environment Variables (.env)

**Backend .env:**

```
MONGO_URI=mongodb+srv://username:password@cluster.mongodb.net/
    soundscape
PORT=5000
NODE_ENV=development
```

## A.3. Project Statistics

| Metric | Count |
|---|---:|
| Backend Routes | 8 |
| API Endpoints | 12 (GET, POST, PUT, DELETE) |
| Frontend Components | 7+ |
| Database Collections | 3 |
| Total Models | 3 |
| Controllers | 3 |
| Total Git Commits | 15 |
| Development Period | 14 Days (Apr 8 – Apr 21, 2026) |

Table 2: Project Statistics

## A.4. Git Commit Timeline

The development followed a logical progression from infrastructure setup through feature implementation to bug fixes and documentation altought not all of them followed the specific format of fix, feat, or refactor.

| Date | Commit Hash | Description |
|---|---|---|
| Apr 8 | `10fb2cd6` | Initial commit |
| Apr 8 | `99dc148e` | npm setup for root, frontend, backend |
| Apr 8 | `1a190a5d` | Following commit for previously non-staged changes |
| Apr 8 | `2884b1a1` | env and .gitignore |
| Apr 8 | `156b5d82` | MongoDB connection (standard string, SRV troubleshoot) |
| Apr 8 | `e6af04ce` | Routes, controllers, server.js |
| Apr 8 | `94b6149e` | Frontend views, Socket.IO, package.json update |
| Apr 8 | `27e5a343` | README update |
| Apr 20 | `dec50195` | .env.example for good practice |
| Apr 20 | `ee3146c6` | PUT endpoint, ObjectId validation, edit UI |
| Apr 20 | `566e1fa9` | ERD diagram generated |
| Apr 21 | `3cacae2a` | Bug fixes: roomController, queueController, audio cleanup |
| Apr 21 | `3b3a0a2f` | Problem stated in README |
| Apr 21 | `c0825154` | README update |
| Apr 21 | `800a1811` | LaTeX report first version (buggy) |

Table 3: Development Timeline